Solution: Pair with Target Sum

Problem Statement

Given an array of numbers sorted in ascending order and a target sum, find a **pair in the array whose sum is equal to the given target.** If no such pair exists return [-1, -1].

Write a function to return the indices of the two numbers (i.e. the pair) such that they add up to the given target.

Example 1:

```
Input: [1, 2, 3, 4, 6], target=6
Output: [1, 3]
Explanation: The numbers at index 1 and 3 add up to 6: 2+4=6
```

Example 2:

```
Input: [2, 5, 9, 11], target=11
Output: [0, 2]
Explanation: The numbers at index 0 and 2 add up to 11: 2+9=11
```

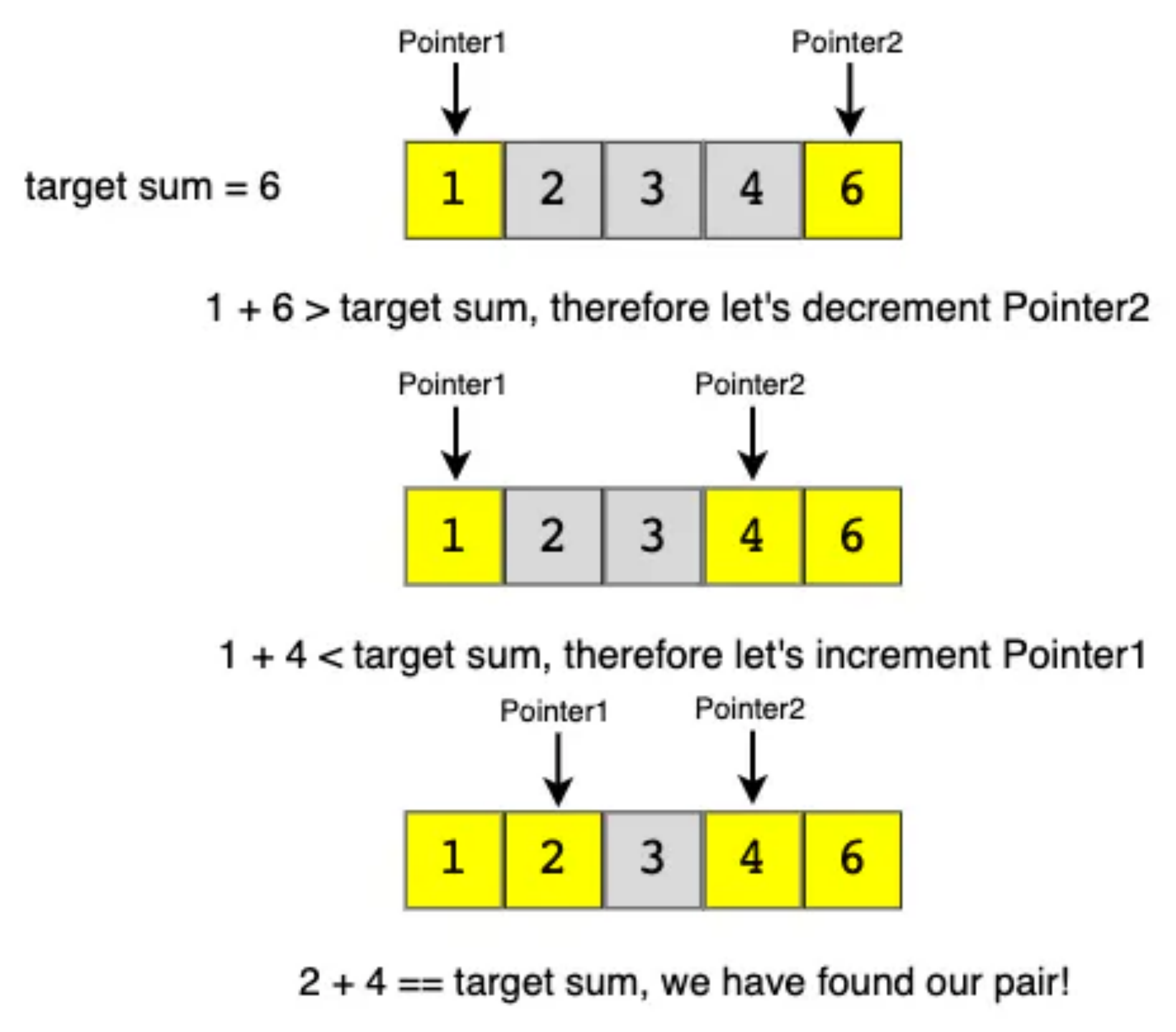
Solution

Since the given array is sorted, a brute-force solution could be to iterate through the array, taking one number at a time and searching for the second number through **Binary Search**. The time complexity of this algorithm will be $O(N * logN)$. Can we do better than this?

We can follow the **Two Pointers** approach. We will start with one pointer pointing to the beginning of the array and another pointing at the end. At every step, we will see if the numbers pointed by the two pointers add up to the target sum. If they do, we have found our pair; otherwise, we will do one of two things:

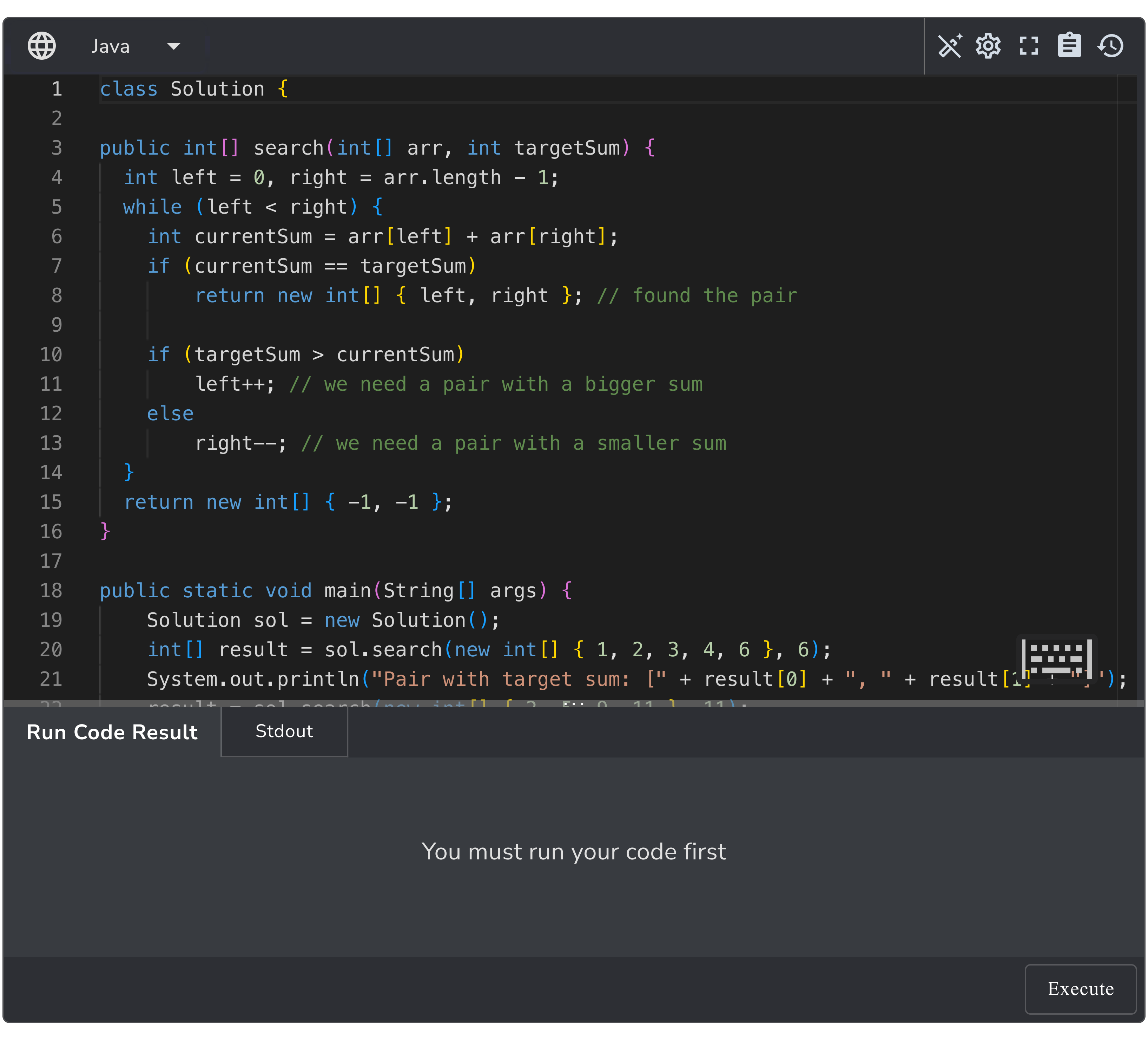
1. If the sum of the two numbers pointed by the two pointers is greater than the target sum, this means that we need a pair with a smaller sum. So, to try more pairs, we can decrement the end-pointer.
2. If the sum of the two numbers pointed by the two pointers is smaller than the target sum, this means that we need a pair with a larger sum. So, to try more pairs, we can increment the start-pointer.

Here is the visual representation of this algorithm for Example-1:


Pointer1
Pointer2
target sum = 6
1
2
3
4
6
1 + 6 > target sum, therefore let's decrement Pointer2
Pointer1
Pointer2
1
2
3
4
6
1 + 4 < target sum, therefore let's increment Pointer1
Pointer1
Pointer2
1
2
3
4
6
2 + 4 == target sum, we have found our pair!

Code

Here is what our algorithm will look like:



```
class Solution {

public int[] search(int[] arr, int targetSum) {
  int left = 0, right = arr.length - 1;
  while (left < right) {
    int currentSum = arr[left] + arr[right];
    if (currentSum == targetSum)
        return new int[] { left, right }; // found the pair

    if (targetSum > currentSum)
        left++; // we need a pair with a bigger sum
    else
        right--; // we need a pair with a smaller sum
  }
  return new int[] { -1, -1 };
}

public static void main(String[] args) {
    Solution sol = new Solution();
    int[] result = sol.search(new int[] { 1, 2, 3, 4, 6 }, 6);
    System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
```

You must run your code first

Time Complexity

1. **Initialization**: Constant time, $O(1)$, as it involves assigning values to `left` and `right`.
2. **While Loop**:
   - The `while` loop runs as long as `left` is less than `right`.
   - In the worst case, this loop iterates over all elements of the array once. This happens when no pair is found, or the pair is found at the extreme ends of the array.
   - Each iteration involves a constant amount of work: calculating `currentSum`, comparing it with `targetSum`, and then incrementing `left` or decrementing `right`.

   Therefore, the loop runs in $O(n)$ time, where n is the number of elements in the array.
3. **Overall**: The dominant factor in this algorithm is the while loop, making the overall time complexity $O(n)$.

Space Complexity

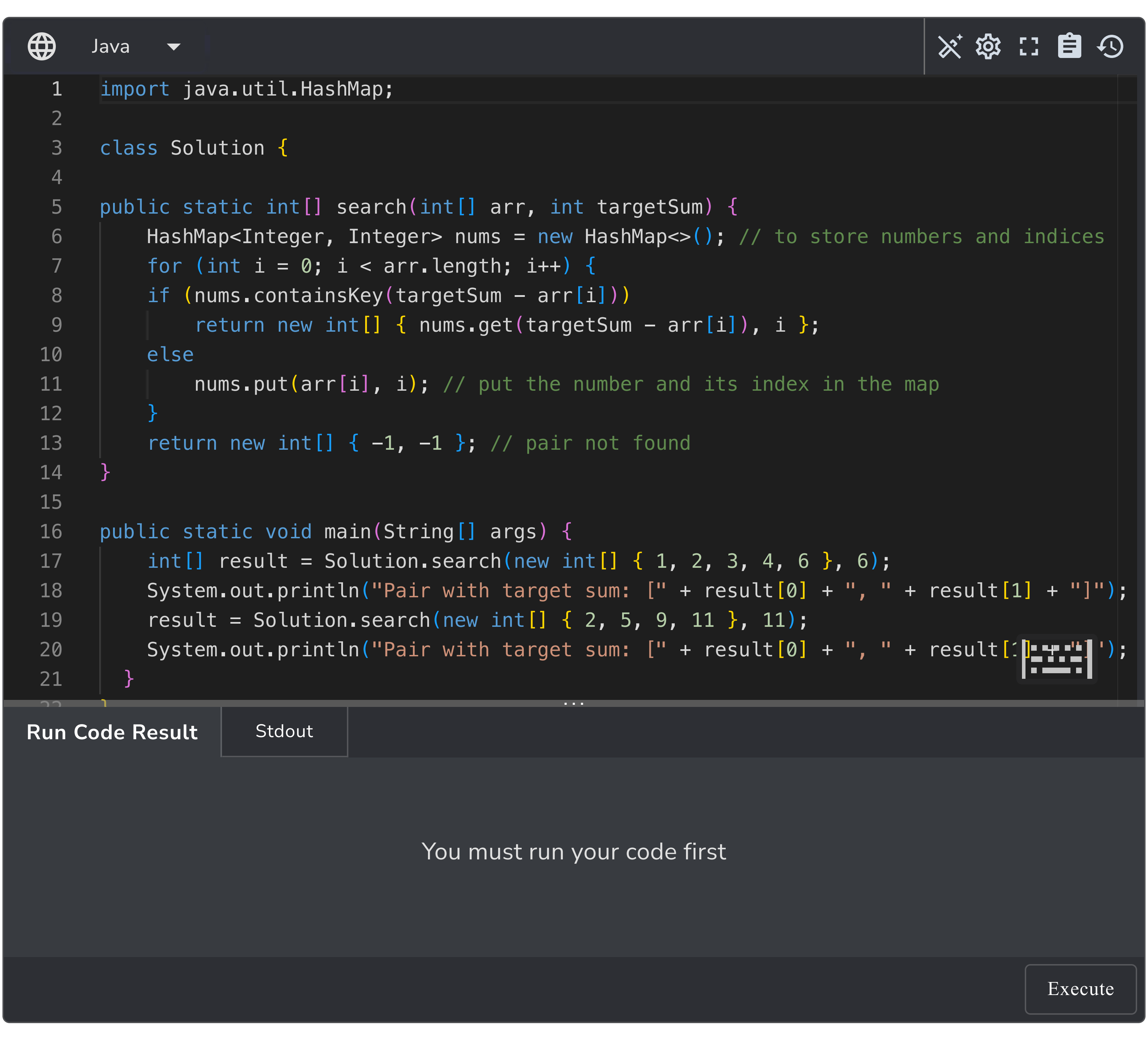
- The algorithm uses a fixed amount of extra space: variables `left`, `right`, and `currentSum`.
- It does not depend on the size of the input array, as no additional data structures are used that grow with the input size.
- Thus, the space complexity is $O(1)$, constant space.

In summary, the algorithm has a time complexity of $O(n)$ and a space complexity of $O(1)$.

An Alternate approach

Instead of using a two-pointer or a binary search approach, we can utilize a HashTable to search for the required pair. We can iterate through the array one number at a time. Let's say during our iteration we are at number 'X', so we need to find 'Y' such that "$X + Y == Target$". We will do two things here:

Search for 'Y' (which is equivalent to "Target−X") in the HashTable. If it is there, we have found the required pair. Otherwise, insert "X" in the HashTable, so that we can search it for the later numbers. Here is what our algorithm will look like:



```
import java.util.HashMap;

class Solution {

public static int[] search(int[] arr, int targetSum) {
    HashMap<Integer, Integer> nums = new HashMap<>(); // to store numbers and indices
    for (int i = 0; i < arr.length; i++) {
    if (nums.containsKey(targetSum - arr[i]))
        return new int[] { nums.get(targetSum - arr[i]), i };
    else
        nums.put(arr[i], i); // put the number and its index in the map
    }
    return new int[] { -1, -1 }; // pair not found
}

public static void main(String[] args) {
    int[] result = Solution.search(new int[] { 1, 2, 3, 4, 6 }, 6);
    System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
    result = Solution.search(new int[] { 2, 5, 9, 11 }, 11);
    System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
  }
```

You must run your code first

Time Complexity

1. **HashMap Initialization**: Constant time, (O(1)).
2. **For Loop**:
   - The `for` loop iterates over each element of the array once.
   - In each iteration, the algorithm checks if the element is already present in the `HashMap` (or `dictionary` in Python) and either returns a result or inserts an element into the `HashMap`.
   - This element checking or insertion operations of a `HashMap` generally operate in (O(1)) time due to efficient hashing. However, in the worst case (e.g., when many hash collisions occur), these operations can degrade to (O(n)). Under the assumption of a good hash function with minimal collisions, these operations can be considered (O(1)).

   Therefore, the loop runs in (O(n)) time in the average case, where (n) is the number of elements in the array.
3. **Overall**: The dominating factor in this algorithm is the for loop. Under the assumption of efficient hashing, the overall average time complexity is (O(n)).

Space Complexity

- The algorithm uses a `HashMap` to store elements from the array. In the worst case, this map can store all elements of the array if no pair is found that adds up to the target sum
- Thus, the space complexity is proportional to the number of elements in the array, which is (O(n)).

In summary, the algorithm has an average time complexity of (O(n)) and a space complexity of (O(n)). The time complexity can degrade to (O(n^2)) in the worst case due to potential hash collisions, but this is generally not the common case with a good hash function.

← Previous
Pair With Target Sum (Easy)
Next →
Buy Two Chocolates (Easy)
☐ Mark as Completed